

1. Selected Area

Computer Architecture / Digital Design

2. Problem Definition

Note: This is the human-readable problem definition. After experimenting with many API calls and different prompting techniques, I had to modify the details to achieve the desired success rate and the exact prompt is in prompts.py.

Implement a DMA scatter-gather controller in verilog. The LLM will receive an interface spec, descriptor format, state machine requirements, and general requirements. The prompt is in the repository under prompt.py.

This is a mechanism that allows a system to move data between different blocks of memory and a stream of data without CPU intervention.

Deliverables:

- 1) Verilog code
- 2) Logic explanation

3. Success Criteria (Testing-Driven)

- 1) Compiles successfully with Icarus Verilog
- 2) Simulation must run in vvp without errors
- 3) Data from addresses 16-19 must be correctly transferred to 32, 36, 40, and 44

4. Automatic Verifier Testcases

Uploaded to the repository.

Pass: passcase.v

Fail: failcase.v

=== Test Case 1 (Expected PASS) ===

Pass: True

Reason:

Analysis: {'compilation': 'OK', 'simulation': 'OK', 'dma_test': 'PASSED'}

Hints: []

=== Test Case 2 (Expected FAIL) ===

Pass: False

Reason: DMA test failed

Analysis: {'compilation': 'OK', 'simulation': 'OK', 'dma_test': 'FAILED'}

Hints: ['No write operations occurred']

5. LLM API Pipeline

Pipeline code attached in repository

6. Repeated Sampling

LLM	Trial	Pass/Fail
GPT 5.4	1	Fail
	2	Fail
	3	Fail
	4	Pass
	5	Pass

Detailed Failure Reasons:

Failure Reason: DMA test failed - no memory reads

Failure Reason: DMA test failed - no write operations

Failure Reason: DMA test failed - no write operations

7. Self-Refinement

LLM	Turn 1	Turn 2	Turn 3	Pass / Fail
GPT-5.4	Fail	Fail	Fail	Fail

It looks like self-refinement did not work for this problem. I noticed that for the repeated sampling test, the LLM would succeed on occasion when running independent trials. However, for this step, it seems that once the LLM comes up with a result it gets stuck and even feedback/hints didn't help it fix the problematic code. I even tried making error messages more descriptive, but the model still struggled. I think the model also struggled with associating these error messages with specific parts of the code.

8. Final Analysis

1. The main failures I saw at first were a lot of compilation errors, since the model struggled to produce even basic compilable verilog code from the problem statement. As such, I had to pass in some boilerplate/syntax into the initial prompt so that I could see the model's performance on logic instead of having it instantly fail the compilation test.
2. The LLM pretty much only made the same error or got the answer correct across all the trials I conducted. Rather than making a different error every time, it seemed to fall into one of a few buckets with the majority being failures. Before I added starter boilerplate code to the prompt, almost every response failed the compilation check.
3. The instructor enforcement worked pretty well for me, and I didn't face any parsing issues. That's also because my use case was simple though, where the deliverable was a single code file that I then had the verifier compile and simulate.
4. I think one thing we could do is explicitly validate some of the schemas in the verilog code. A lot of errors came from compilation or missing signals / logic, so forcing the LLM to provide this would probably make a big difference. Right now, I had one pydantic field for the whole verilog script. Instead, maybe I could have had a field for each TODO block so that the model could focus on sub-components and have a general working solution.
5. See above with regards to richer pydantic fields